Cyprus University of Technology

# CSE 227 Προχωρημένη Αντικειμενοστρεφής Μεθοδολογία και Προγραμματισμός (UML, Java)

## Εισαγωγή

Παναγιώτης Ηλία

# Συστάσεις

- Διδάσκων: Παναγιώτης Ηλία
  - E-mail: panagiotis.ilia@cut.ac.cy
  - Γραφείο: 420, Τμήμα ΗΜΜΗΥΠ
  - Ενδιαφέροντα:
    - Κυβερνοασφάλεια και Ιδιωτικότητα
    - Κρυπτογραφία
    - Κοινωνικά δίκτυα
  - Μαθήματα:
    - CSE 227 Προχωρημένη Αντικειμενοστρεφής Μεθοδολογία και Προγραμματισμός (UML, Java)
    - CSE 423 Εισαγωγή στην Κρυπτογραφία και Ασφάλεια Υπολογιστών
    - CSE 529 Ασφάλεια και Ιδιωτικότητα στην Τεχνητή Νοημοσύνη και τα Συστήματα Δεδομένων

# Πληροφορίες

- Διαλέξεις
  - Τρίτη 10:30 - 12:30, Κτήριο Τάσσος Παπαδόπουλος, Αμφιθέατρο 1
  - Πέμπτη 13:30 - 15:30, Κτήριο Τάσσος Παπαδόπουλος, Αίθουσα Διαλέξεων 3
- Εργαστήρια
  - Δευτέρα 15:30 - 17:30, Κτήριο ΗΜΜΗΥΠ
  - Θα χωριστείτε σε 4 groups (1 ώρα το κάθε group)
  - Τα εργαστήρια θα ξεκινήσουν την 5η εβδομάδα

**Οι διαλέξεις και τα εργαστήρια είναι υποχρεωτικά**

- Ώρες Γραφείου
  - Τρίτη 12:30 - 14:30
  - Για διευθέτηση συνάντησης εκτός ωρών γραφείου επικοινωνήστε μέσω email

# Υλικό Μαθήματος

- Σελίδα μαθήματος στην πλατφόρμα elearning
  - **https//elearning.cut.ac.cy**
  - Οι σημειώσεις των διαλέξεων και το υλικό των εργαστηρίων, όπως επίσης οι εργασίες του μαθήματος και οι ανακοινώσεις, θα αναρτώνται στην σελίδα του μαθήματος

# Βαθμολόγηση

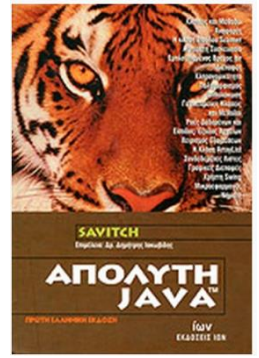
- **45% Τελική Εξέταση (εφ' όλης της ύλης)**
- **25% Ενδιάμεση Εξέταση**
- **20% Εργασία**
- **10% Εργαστήρια**

- **Απαραίτητες προϋποθέσεις επιτυχίας στο μάθημα είναι:**
  - Η εξασφάλιση συνολικού βαθμού πέραν του **50% στο μάθημα**
  - Η εξασφάλιση συνολικού βαθμού μεγαλύτερου ή ίσου με **45% στην τελική εξέταση**
    - **Σε περίπτωση μη εξασφάλισης βαθμού μεγαλύτερου ή ίσου με 45% στην τελική εξέταση, ο βαθμός που θα περαστεί στο σύστημα θα είναι ΜΟΝΟ ο βαθμός της Τελικής Εξέτασης**

- ***Ο διδάσκων διατηρεί το δικαίωμα μικρών αλλαγών στην πιο πάνω κατανομή της βαθμολογίας***

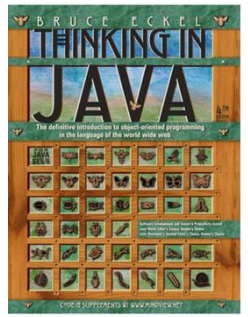
# Βιβλιογραφία

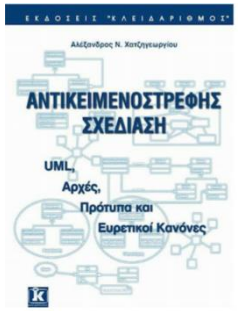

- "Απόλυτη JAVA" του Savitch Walter, Εκδόσεις Ίων, 2008
- "JAVA, μια εισαγωγή στην επίλυση προβλημάτων και στον προγραμματισμό" του Savitch Walter, εκδόσεις Τζιόλα
- "Thinking in Java" by Bruce Eckel, 4th edition, Prentice Hall
- "Java Προγραμματισμός", των Harvey M. Deitel και Paul J. Deitel, εκδόσεις Γκιούρδας
- "Αντικειμενοστρεφής σχεδίαση με UML, αρχές, πρότυπα και ευρετικοί κανόνες" του Αλέξανδρου Χατζηγεωργίου, εκδόσεις κλειδάριθμος
- "Introduction to Programming in Java" by Robert Sedgewick and Kevin Wayne, Addison Wesley

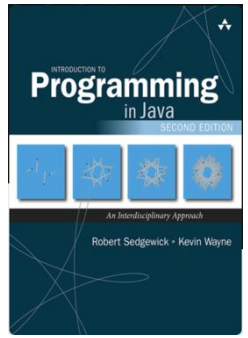



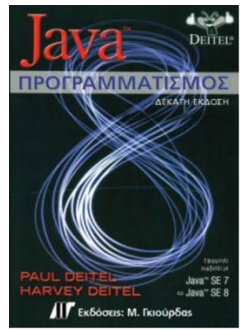






# Χρήσιμο Υλικό

- Οι διαφάνειες του μαθήματος θα αναρτώνται στην πλατφόρμα elearning
- Java Docs: Online documentation της Oracle για τη γλώσσα Java
  - Λεπτομερής περιγραφή για κάθε κλάση και κάθε μέθοδο
- Το Web: Για κάθε προγραμματιστική (ή άλλη) ερώτηση που έχετε μπορείτε να βρείτε απαντήσεις online
  - Π.χ., stackoverflow.com είναι ένα online forum στο οποίο έμπειροι προγραμματιστές απαντάνε σε ερωτήσεις
  - Για κάθε μήνυμα λάθους μπορείτε να βρείτε πληροφορίες για το τι σημαίνει και πως μπορείτε να το λύσετε
- Βοηθάει για να εξοικειωθείτε και με την αγγλική ορολογία, θα την χρησιμοποιούμε κατά καιρούς και στο μάθημα

# Κώδικας Δεοντολογίας




- Η δουλειά που παρουσιάζετε σαν δικιά σας, **να είναι όντως δικιά σας**
  - Μην επιτρέπετε σε άλλους να χρησιμοποιήσουν τον κώδικα σας
  - Μην χρησιμοποιήσετε κώδικα που δεν είναι δικός σας
  - Μην λύνετε τις ασκήσεις με την βοήθεια συναδέλφων σας, **είναι ατομικές**
  - Μην χρησιμοποιήσετε ChatGPT ή άλλο εργαλείο AI **(για τις εργασίες)**

- **Οι εργασίες θα ελέγχουν (με διάφορα εργαλεία)**

- **Παραβίαση των πιο πάνω κανόνων θα θεωρηθεί αντιγραφή**

# Κώδικας Δεοντολογίας




- Συγκεκριμένα, για ChatGPT και άλλα εργαλεία AI
  - Μάθετε να τα χρησιμοποιείτε **σωστά**
    - Για να σας βοηθήσει να καταλάβετε κάποιες έννοιες
    - Για να σας δώσει παραδείγματα
    - **Γενικά, για να σας βοηθήσει να μάθετε**
  - Δεν του ζητάμε λύσεις για ασκήσεις/εργασίες
    - Αν χρησιμοποιηθεί οπουδήποτε (για κάποιο λόγο) πρέπει:
      - Να αναφερθεί ακριβώς σε ποιο σημείο χρησιμοποιήθηκε
      - Πως χρησιμοποιήθηκε (ποιο prompt δώσατε)





# Στόχοι Μαθήματος

- Κατανόηση των βασικών αρχών και τεχνικών του αντικειμενοστρεφή προγραμματισμού και την υλοποίησή τους σε λογισμικό
- Περιγραφή του σχεδίου της λύσης ενός προβλήματος με αντικειμενοστρεφή τρόπο, κάνοντας χρήση διαγραμμάτων UML
- Εξοικείωση με το συντακτικό και το API (βιβλιοθήκες και μέθοδοι) της JAVA, και δυνατότητα επίλυσης πολύπλοκων προβλημάτων με τη JAVA
- Ευχέρεια ανάπτυξης, τεκμηρίωσης, ελέγχου και αποσφαλμάτωσης προγραμμάτων JAVA για την αποδοτική υλοποίηση αλγορίθμων
- Εξοικείωση με Ολοκληρωμένα Περιβάλλοντα Ανάπτυξης (Integrated Development Environments) όπως NetBeans και Eclipse

# Ύλη που θα καλύψουμε

- Εισαγωγή στη Java
  - Βασικό συντακτικό και δομή προγράμματος
  - Είσοδος, έξοδος δεδομένων
  - Εξαιρέσεις
  - Γραφικά
- Αρχές αντικειμενοστρεφούς προγραμματισμού
  - Κλάσεις και αντικείμενα
  - Ενθυλάκωση και απόκρυψη
  - Πολυμορφισμός και Κληρονομικότητα
  - Αφηρημένες κλάσεις, διεπαφές (Interfaces)
- Αντικειμενοστρεφής σχεδίαση και διαγράμματα UML
  - Διαγράμματα κλάσεων, ροής γεγονότων, περιπτώσεων χρήσης
  - Πρότυπα καλής δόμησης λογισμικού και απεικόνιση σε UML σχεδιασμό

## Ευχαριστίες

- Πρωτότυπο υλικό Παναγιώτης Τσαπάρας και Παναγιώτης Ανδρέου
- Τροποποιήσεις, επεκτάσεις και διορθώσεις Παναγιώτης Ηλία

# CEI 227 Προχωρημένη Αντικειμενοστρεφής Μεθοδολογία και Προγραμματισμός (UML, Java)

## Εισαγωγή στη Java

# Προγράμματα

- Ένα πρόγραμμα (software) είναι ένα σύνολο από οδηγίες/εντολές προς τον υπολογιστή
- Δεδομένου ότι ο υπολογιστής δεν καταλαβαίνει την φυσική γλώσσα αλλά μόνο την γλώσσα μηχανής (machine language), χρειαζόμαστε κάποιες ειδικές γλώσσες (γλώσσες προγραμματισμού) για να επικοινωνήσουμε μαζί του.
- Παραδείγματα γλωσσών προγραμματισμού: C, C++, JAVA, C#, κ.τ.λ.

ΠΑΡΑΔΕΙΓΜΑ ΠΡΟΓΡΑΜΜΑΤΟΣ σε JAVA

```
public class HelloWorld{

   public static void main(String[] args){
      System.out.println("Hello World!!!");
   }
}
```

# Γλώσσες Προγραμματισμού



- Πρώτη γενιά: Γλώσσα μηχανής

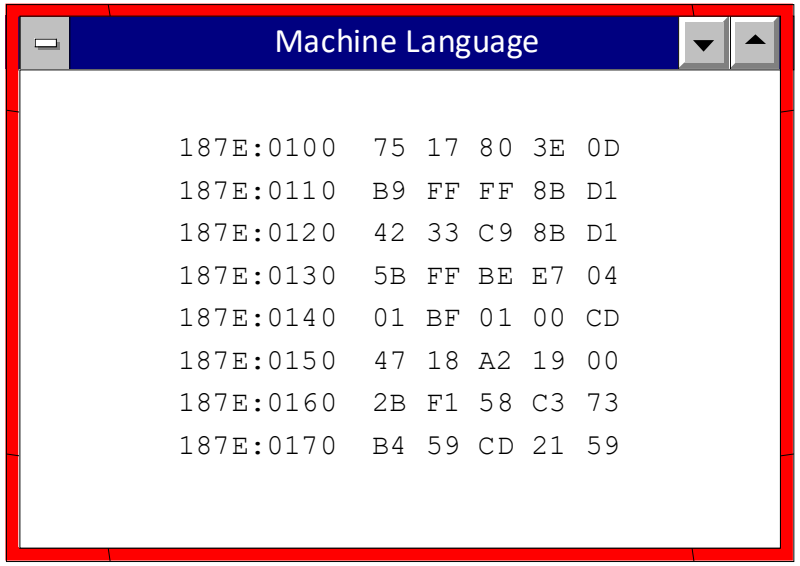
Ο προγραμματιστής μετατρέπει το πρόβλημα του σε ένα πρόγραμμα

- Π.χ. πώς να υπολογίσω το μέγιστο κοινό διαιρέτη δύο αριθμών

Γράφει ακριβώς τις εντολές που θα πρέπει να εκτελέσει ο υπολογιστής

- Θα πρέπει να ξέρει ακριβώς την δυαδική αναπαράσταση των εντολών.

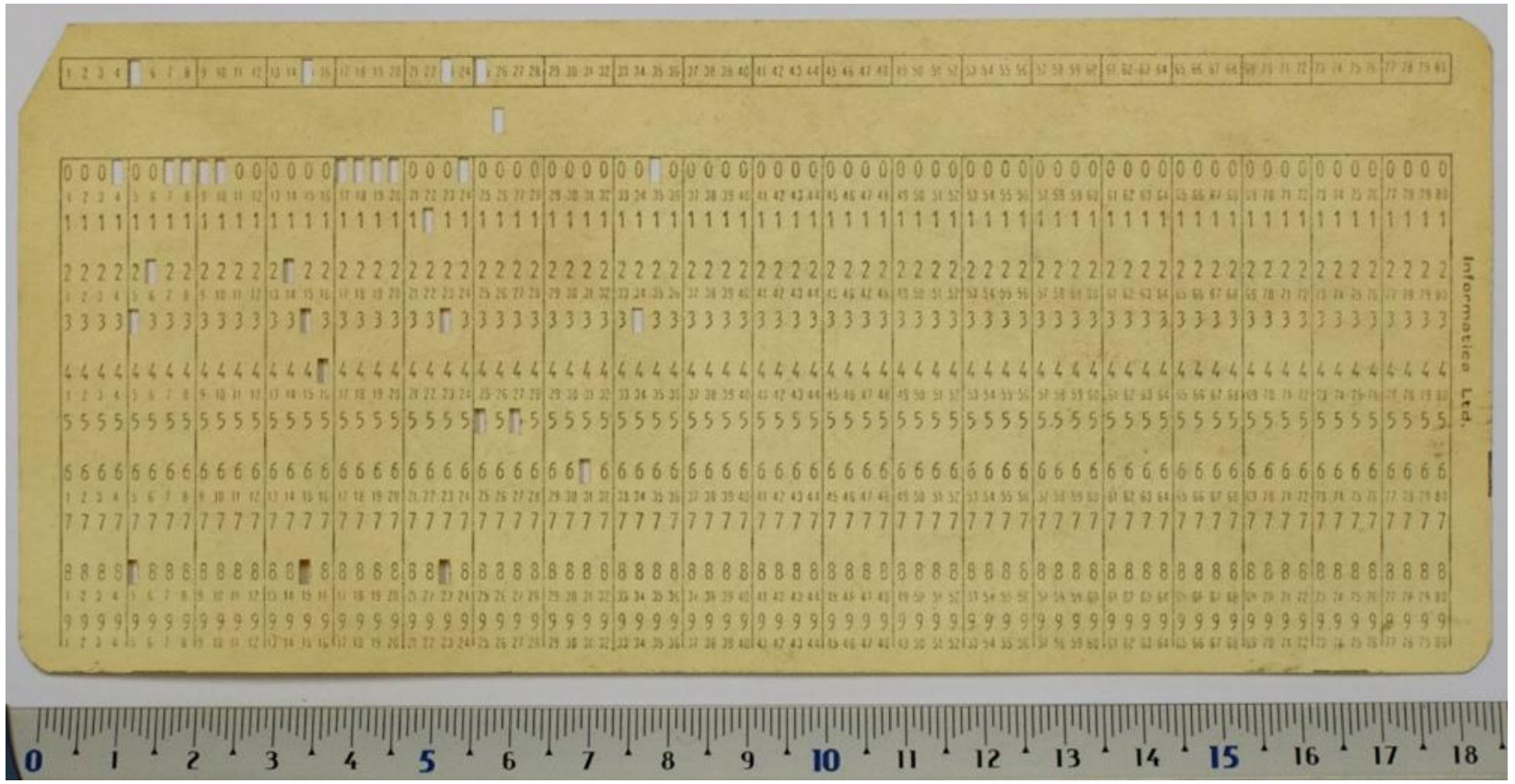
Στους πρώτους υπολογιστές οι εντολές αποστέλλονταν απευθείας στον υπολογιστή με τη χρήση διακοπτών ή κωδικοποιούνταν σε διάτρητες κάρτες




**Program entered and executed as machine language**

# Γλώσσες Προγραμματισμού

- Πρώτη γενιά: Γλώσσα μηχανής

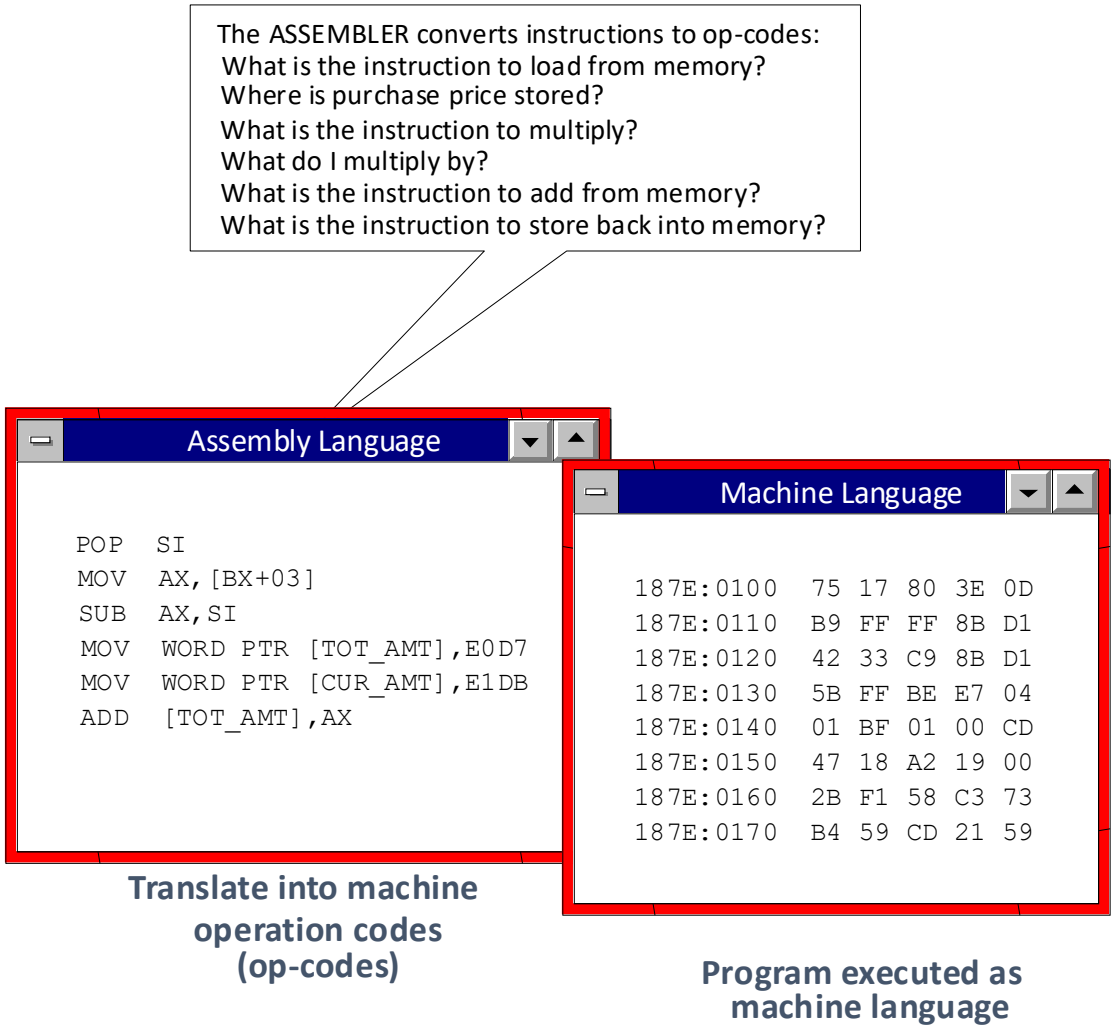


# Γλώσσες Προγραμματισμού

- Πρώτη γενιά: Γλώσσες μηχανής
- Δεύτερη γενιά: Γλώσσες Assembly

Ο προγραμματιστής δεν χρειάζεται να ξέρει ακριβώς την δυαδική αναπαράσταση των εντολών

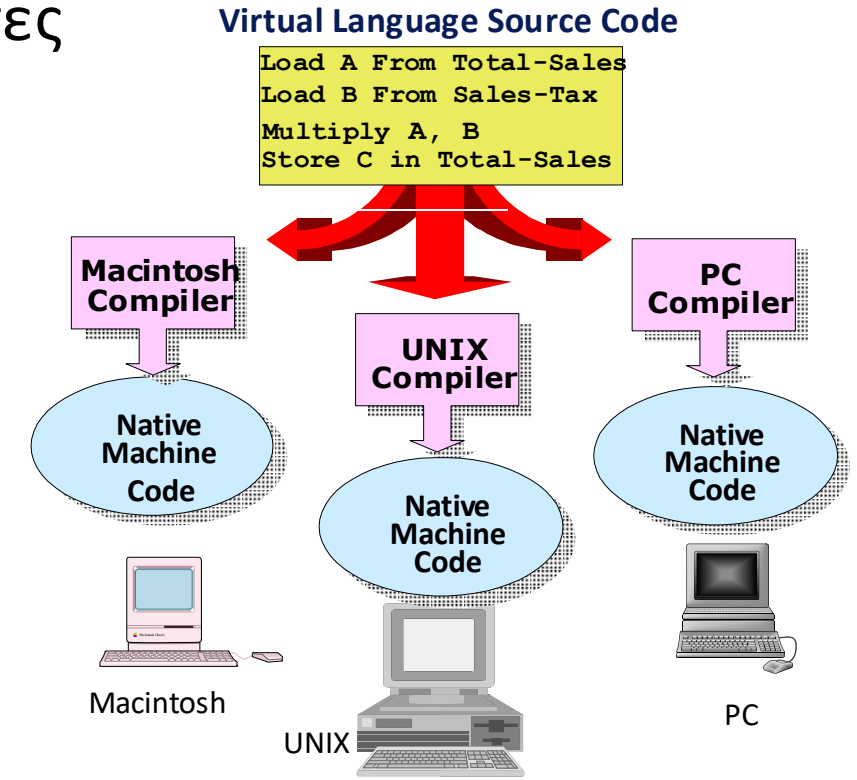
- Χρησιμοποιεί πιο κατανοητούς μνημονικούς κανόνες
- Η γλώσσα είναι ειδικευμένη σύμφωνα με τον επεξεργαστή και περιβάλλον Η/Υ
- Ο Assembler μετατρέπει τα σύμβολα σε γλώσσα μηχανής



# Γλώσσες Προγραμματισμού

- Πρώτη γενιά: Γλώσσες μηχανής
- Δεύτερη γενιά: Γλώσσες Assembly
- Τρίτη γενιά: Υψηλού επιπέδου (high-level) γλώσσες

- Ο προγραμματιστής δίνει εντολές σε μια κατανοητή και καλά δομημένη γλώσσα (πηγαίος κώδικας)
- Ο πηγαίος κώδικας πρέπει να μεταφραστεί από ένα μεταγλωττιστή (compiler)
  - Ο compiler μετατρέπει τις εντολές (πηγαίος κώδικας) σε ενδιάμεσο/εκτελέσιμο κώδικα
  - Ο ενδιάμεσος κώδικας μετατρέπεται σε γλώσσα μηχανής
- Παραδείγματα: C, C++, Pascal, Basic, JAVA



# Γλώσσες Προγραμματισμού

- Πρώτη γενιά: Γλώσσες μηχανής
- Δεύτερη γενιά: Γλώσσες Assembly
- Τρίτη γενιά: Υψηλού επιπέδου (high-level) γλώσσες

The **COMPILER** translates:
Load the purchase price
Multiply it by the sales tax
Add the purchase price to the result
Store the result in total price

**High-Level Language**

```
salesTax = purchasePric * TAX_RATE;
totalSales = purchasePrice + salesTax;
```

**Translate into the instruction set**

**Assembly Language**

```
POP   SI
MOV   AX,[BX+03]
SUB   AX,SI
MOV   WORD PTR [TOT_AMT],E0D7
MOV   WORD PTR [CUR_AMT],E1DB
ADD   [TOT_AMT],AX
```

**Translate into machine operation codes (op-codes)**

**Machine Language**

```
187E:0100   75 17 80 3E 0D
187E:0110   B9 FF FF 8B D1
187E:0120   42 33 C9 8B D1
187E:0130   5B FF BE E7 04
187E:0140   01 BF 01 00 CD
187E:0150   47 18 A2 19 00
187E:0160   2B F1 58 C3 73
187E:0170   B4 59 CD 21 59
```

**Program executed as machine language**

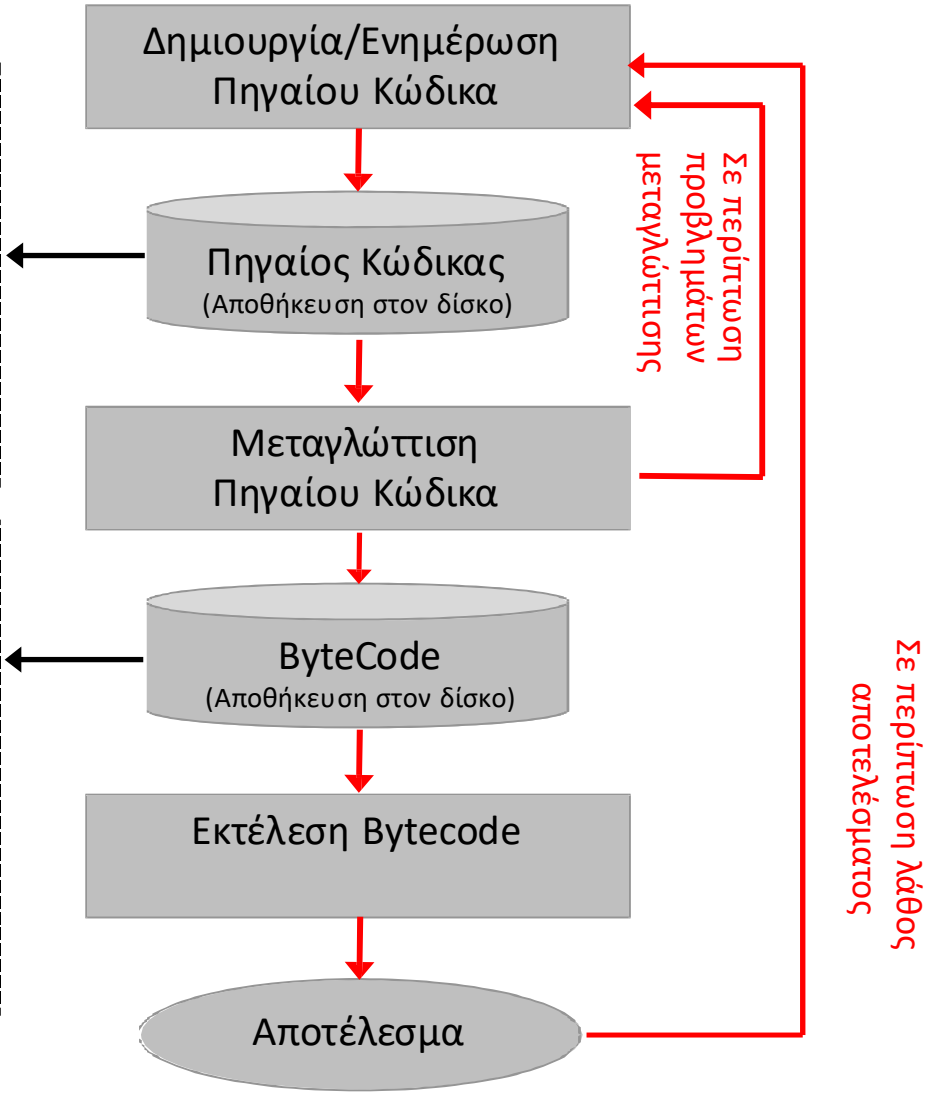
# Τυπικός κύκλος ζωής ενός προγράμματος JAVA

ΠΑΡΑΔΕΙΓΜΑ ΠΗΓΑΙΟΥ ΚΩΔΙΚΑ

```
public class HelloWorld{
   public static void main(String[] args){
      System.out.println("Hello World!!!");
   }
}
```

ΠΑΡΑΔΕΙΓΜΑ BYTE CODE

```
…
Method HelloWorld()
  0       aload_0
…
Method void main(java.lang.String[]){
  0  getstatic #2 …
  3  ldc #3 < String "Hello World!!!" >
  5  invokevirtual #4
…
```



# Γλώσσες Προγραμματισμού

- Πρώτη γενιά: Γλώσσες μηχανής
- Δεύτερη γενιά: Γλώσσες Assembly
- Τρίτη γενιά: Υψηλού επιπέδου (high-level) γλώσσες
- Τέταρτη γενιά: Εξειδικευμένες γλώσσες

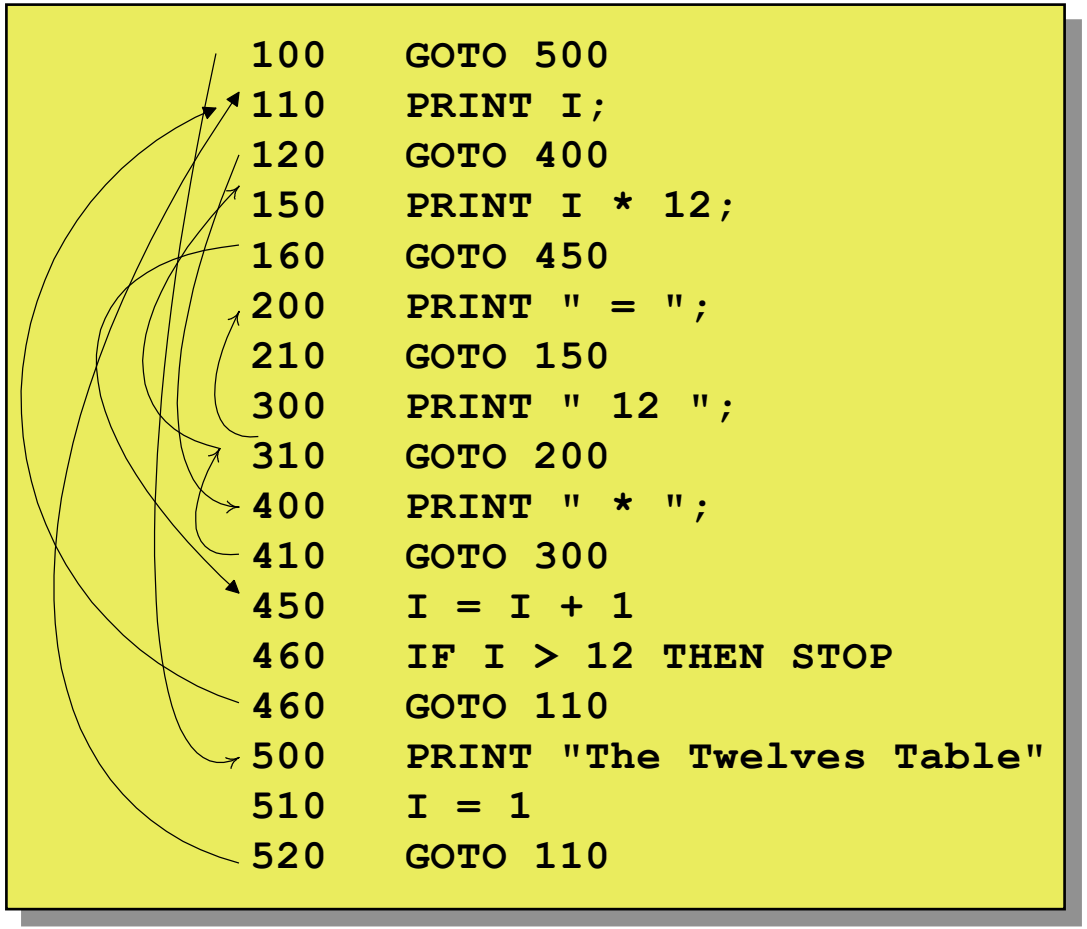
  - Ο προγραμματιστής περιγράφει τι πρέπει να γίνει, όχι πως να γίνει
  - Εξειδικευμένες, κυρίως domain-specific, όπως SQL

**Κάθε γενιά προσθέτει ένα επίπεδο αφαίρεσης.**

# Προγραμματιστικά Υποδείγματα (paradigms)

Προγραμματισμός των πρώτων ημερών.

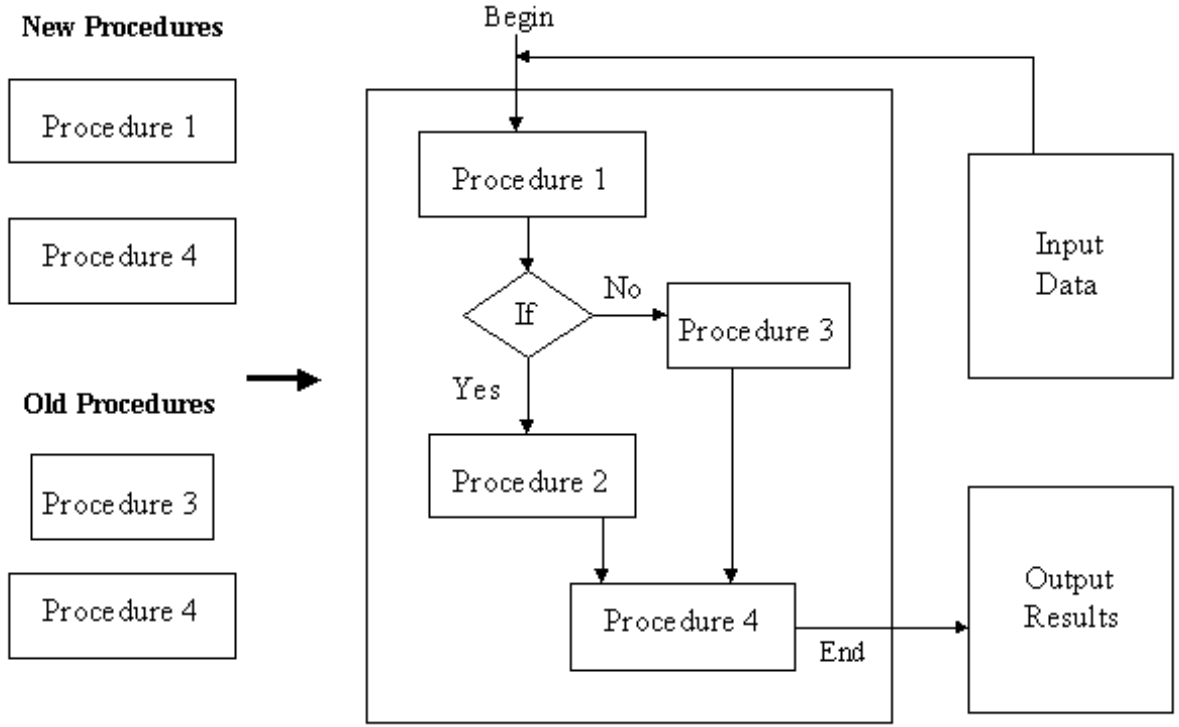
- Spaghetti code
  - Δύσκολο να διαβαστεί και να κατανοηθεί η ροή του



# Προγραμματιστικά Υποδείγματα (paradigms)

## Δομημένος Προγραμματισμός

- Τέσσερεις προγραμματιστικές δομές
  - Sequence – ακολουθιακές εντολές
  - Selection – διακλάδωση με if-then-else
  - Iteration – δημιουργία βρόγχων
  - Recursion – αναδρομή
- Ο κώδικας σπάει σε λογικά blocks που έχουν ένα σημείο εισόδου και εξόδου
  - Κατάργηση της GOTO εντολής
- Οργάνωση του κώδικα σε διαδικασίες (procedures)



# Προγραμματιστικά Υποδείγματα (paradigms)

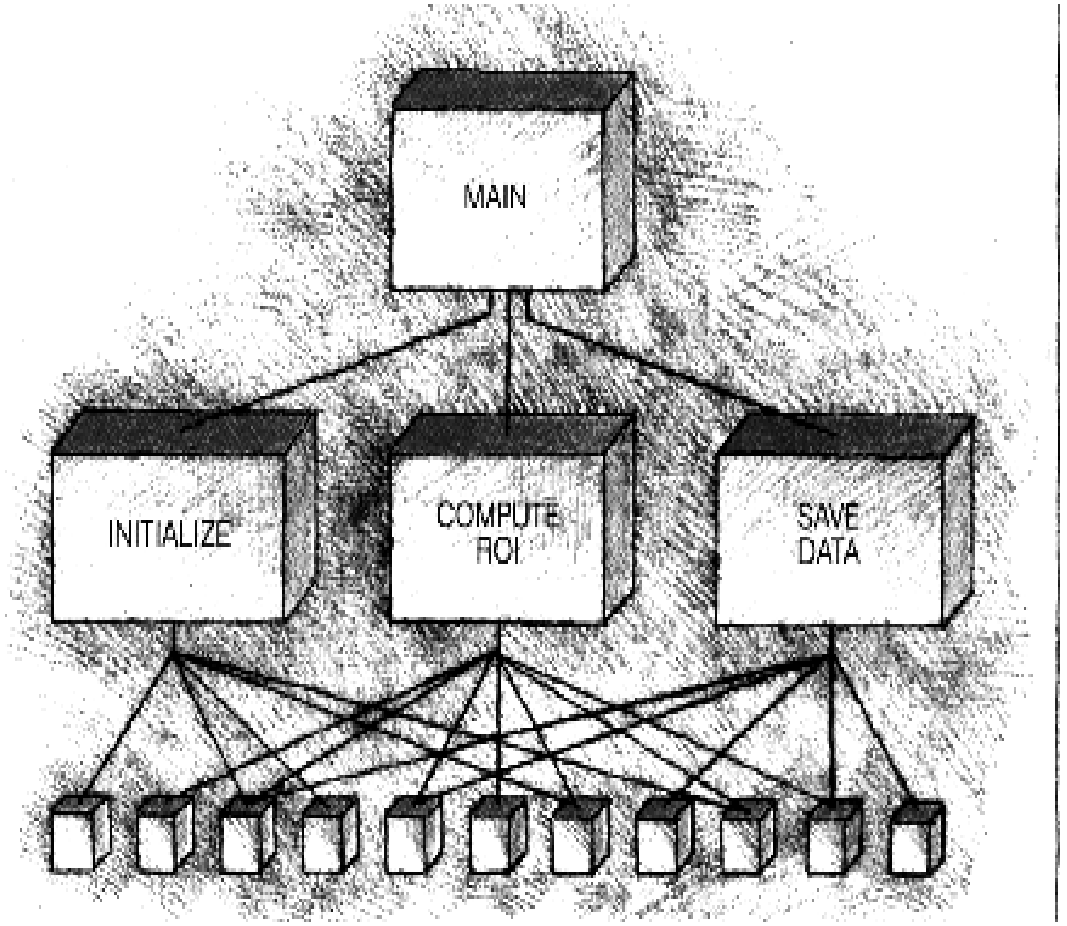
Διαδικαστικός Προγραμματισμός

- Το πρόγραμμα μας σπάει σε πολλαπλές διαδικασίες
  - Κάθε διαδικασία λύνει ένα υπο-πρόβλημα και αποτελεί μια λογική μονάδα (module)
  - Μια διαδικασία μπορούμε να την επαναχρησιμοποιήσουμε σε διαφορετικά δεδομένα
- Το πρόγραμμα μας είναι τμηματοποιημένο (modular)

# Διαδικαστικός Προγραμματισμός

Κοινά Δεδομένα

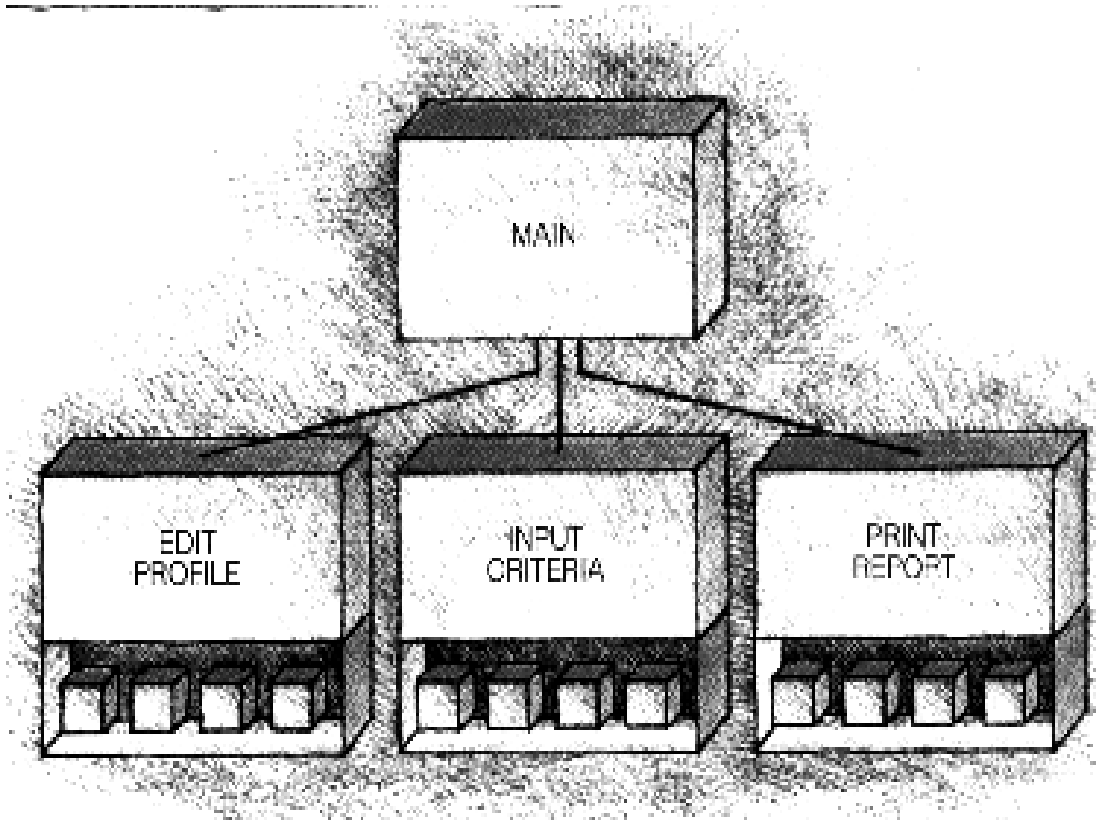
- Ο διαδικαστικός προγραμματισμός τμηματοποιεί τον κώδικα αλλά όχι απαραίτητα τα δεδομένα
- Π.χ. Με τη χρήση καθολικών μεταβλητών (global variables) όλες οι διαδικασίες μπορεί να χρησιμοποιούν τα ίδια δεδομένα και άρα να εξαρτώνται μεταξύ τους.
- Πρέπει να αποφεύγουμε τη χρήση καθολικών μεταβλητών!



# Διαδικαστικός Προγραμματισμός

Απόκρυψη δεδομένων

- Με τη δημιουργία τοπικών μεταβλητών μέσα στις διαδικασίες αποφεύγουμε την ύπαρξη κοινών δεδομένων
- Ο κώδικας γίνεται πιο εύκολο να σχεδιαστεί, να γραφτεί και να συντηρηθεί
- Η επικοινωνία μεταξύ των διαδικασιών γίνεται με ορίσματα.
- Τμηματοποιημένος προγραμματισμός (modular programming)



# Διαδικαστικός Προγραμματισμός

Περιορισμοί του διαδικαστικού προγραμματισμού

- Ο διαδικαστικός προγραμματισμός δουλεύει ΟΚ για μικρά προγράμματα, αλλά για μεγάλα συστήματα είναι δύσκολο να σχεδιάσουμε, υλοποιήσουμε και συντηρήσουμε τον κώδικα
  - Δεν είναι εύκολο να προσαρμοστούμε σε αλλαγές, και δεν μπορούμε να προβλέψουμε όλες τις ανάγκες που θα έχουμε
- **Παράδειγμα:** το πανεπιστήμιο έχει ένα σύστημα για να κρατάει πληροφορίες για φοιτητές και καθηγητές
  - Υπάρχει μια διαδικασία **print** που τυπώνει στοιχεία και βαθμούς φοιτητών
  - Προκύπτει ανάγκη για μια διαδικασία που να τυπώνει τα μαθήματα των καθηγητών
    - Χρειαζόμαστε μια **print2**

# Αντικειμενοστρεφής προγραμματισμός

- Τα προβλήματα αυτά προσπαθεί να αντιμετωπίσει ο αντικειμενοστραφής προγραμματισμός (object-oriented programming)
  - Ο Αντικειμενοστρεφής προγραμματισμός βάζει **μαζί** τα δεδομένα και τις διαδικασίες (μεθόδους) σχετικές με τα δεδομένα
  - Π.χ., ο φοιτητής ή ο καθηγητής έρχεται με μια δικιά του διαδικασία print
- Αυτό επιτυγχάνεται με **αντικείμενα** και **κλάσεις**

# Παράδειγμα

Διαδικαστικός Προγραμματισμός

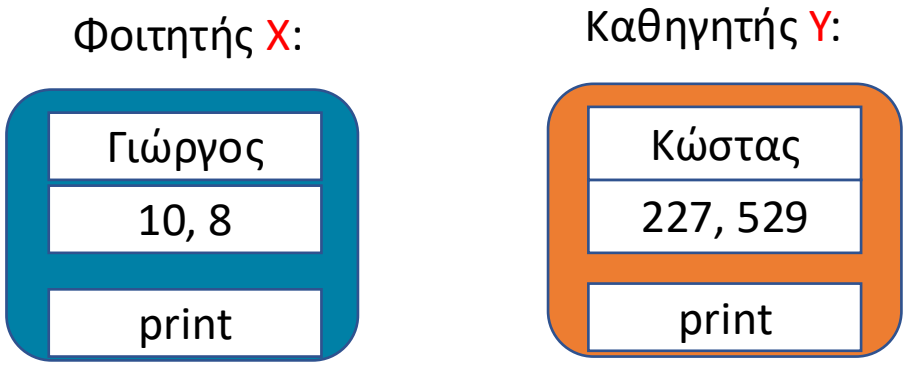


# Παράδειγμα

Διαδικαστικός Προγραμματισμός

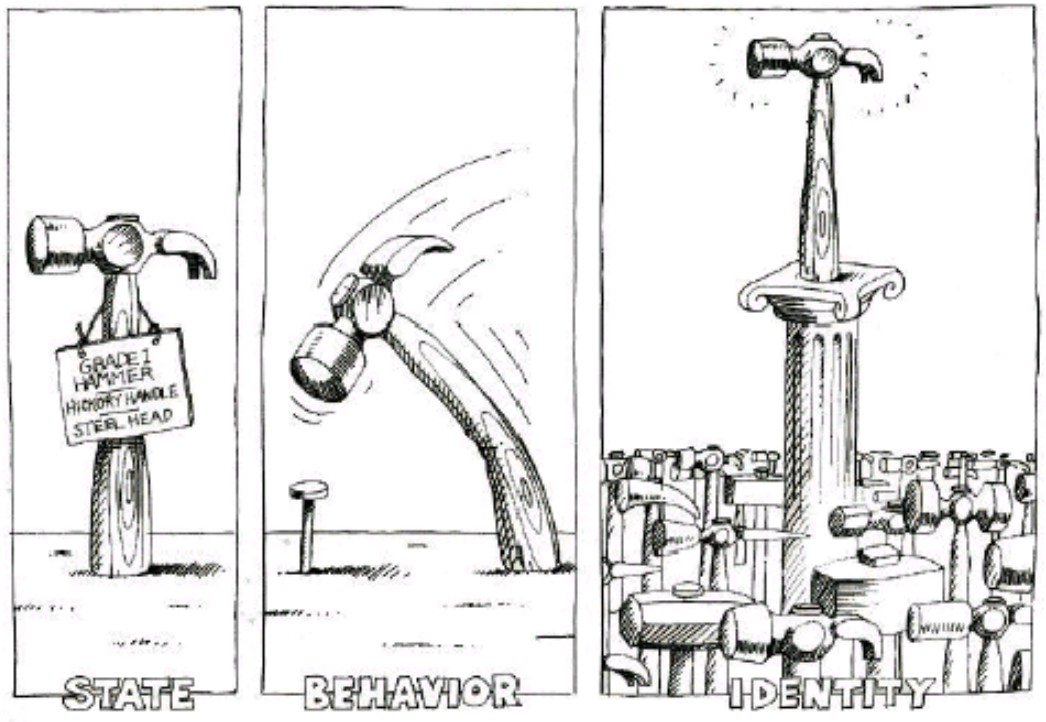



Αντικειμενοστρεφής Προγραμματισμός



# Αντικείμενο

- Ένα αντικείμενο στον κώδικα αναπαριστά μια μονάδα / οντότητα / έννοια η οποία έχει:
  - Μια **κατάσταση**, η οποία ορίζεται από ορισμένα **χαρακτηριστικά**
  - Μια **συμπεριφορά**, η οποία ορίζεται από ορισμένες **ενέργειες** που μπορεί να εκτελέσει το αντικείμενο
  - Μια **ταυτότητα** που το ξεχωρίζει από τα υπόλοιπα αντικείμενα **ίδιου τύπου**

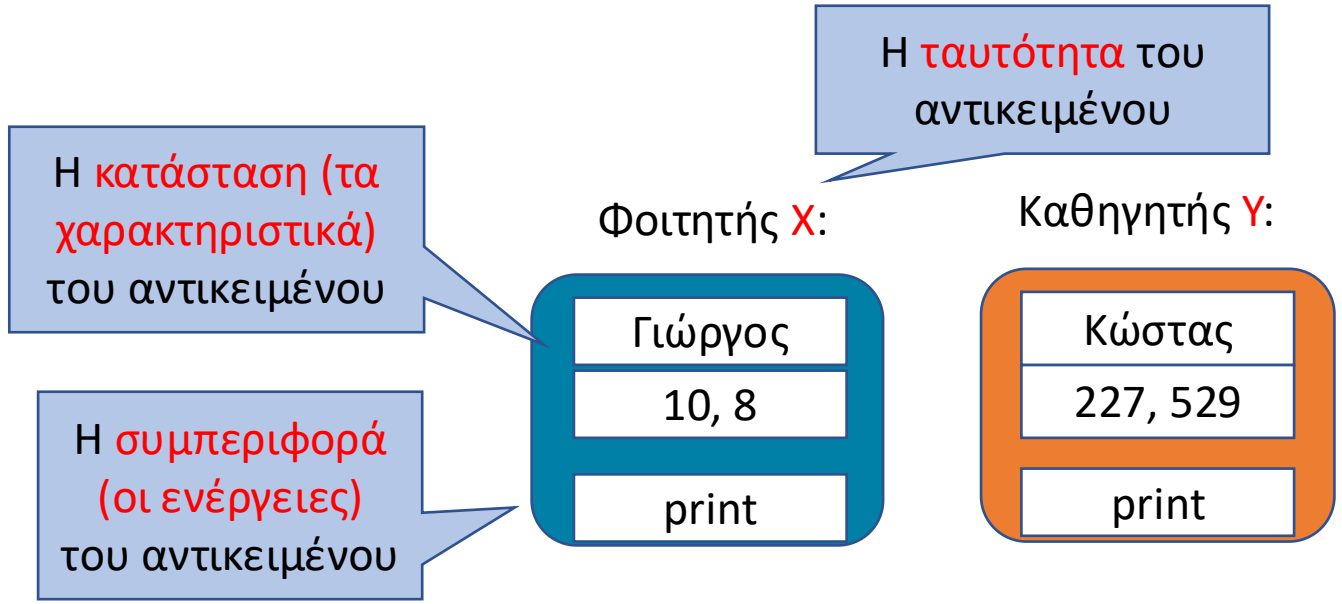


# Παράδειγμα

Διαδικαστικός Προγραμματισμός




Αντικειμενοστρεφής Προγραμματισμός



# Κλάσεις

- Μια αφηρημένη περιγραφή αντικειμένων με κοινά χαρακτηριστικά και κοινή συμπεριφορά
  - Ένα καλούπι που παράγει **αντικείμενα**
  - Ένα **αντικείμενο** είναι ένα **στιγμιότυπο** μίας κλάσης
  - Από μία κλάση μπορούν να δημιουργηθούν πολλά στιγμιότυπα
  - Κάθε κλάση περιγράφει μία μοναδική οντότητα με ξεχωριστές ιδιότητες στο πρόγραμμα

# Κλάσεις

- Η κλάση **φοιτητής** έχει τα γενικά χαρακτηριστικά (όνομα, βαθμοί) και τη συμπεριφορά print
  - Ο **φοιτητής Χ** είναι ένα **αντικείμενο** της **κλάσης φοιτητής**
  - Οι **φοιτητές Χ** και **Υ** είναι δύο στιγμιότυπα της **κλάσης φοιτητής**

- Η κλάση **αυτοκίνητο** έχει τα χαρακτηριστικά (brand, color) και τη συμπεριφορά (drive, stop)
  - Το αυτοκίνητο **ABC123** είναι ένα **αντικείμενο** της **κλάσης αυτοκίνητο** με χαρακτηριστικά (Mazda, black)

# Κλάσεις και Αντικείμενα

Κλάση

Μια αφηρημένη περιγραφή ενός φοιτητή



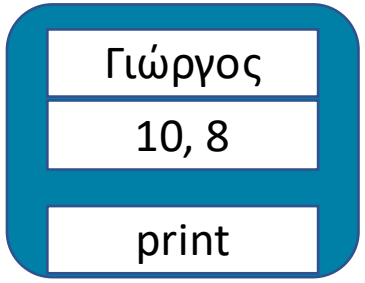

Όνομα κλάσης

Πεδία κλάσης: Ιδιότητες/Χαρακτηριστικά

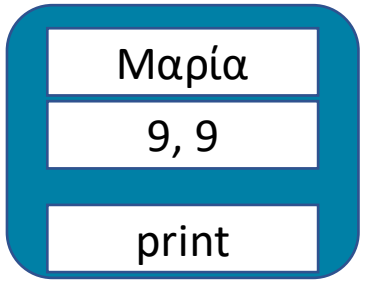
Μέθοδοι κλάσης: Λειτουργίες

Αντικείμενα

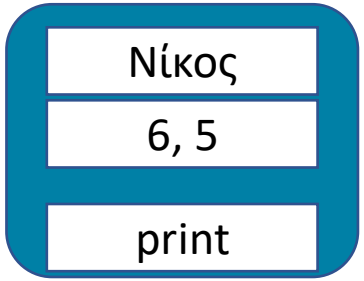
Φοιτητής X:




Φοιτητής Y:



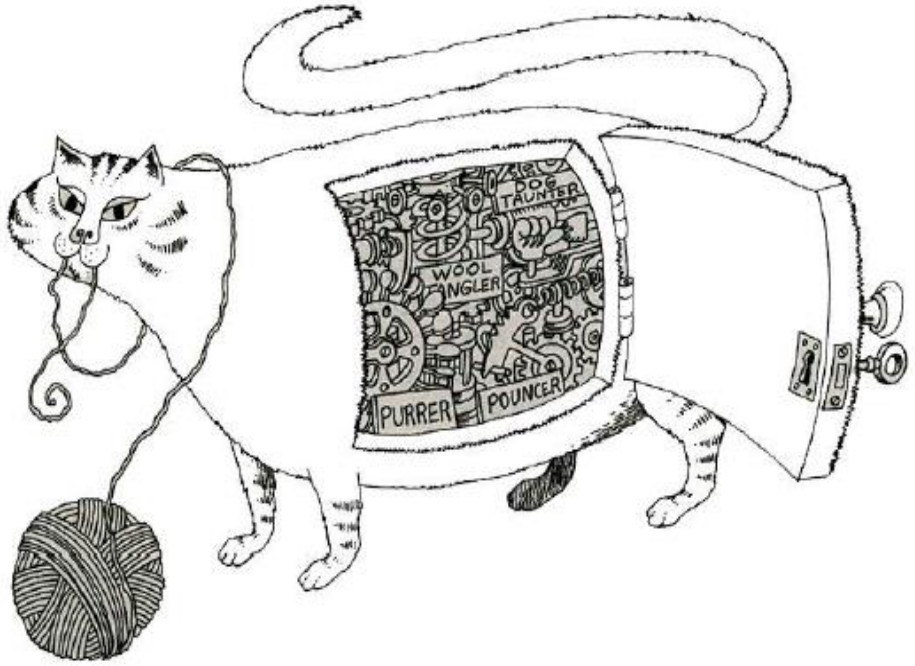

Φοιτητής Z:




Το κάθε αντικείμενο έχει:

- Κατάσταση (το συγκεκριμένο όνομα και βαθμούς)
- Ενέργειες (λειτουργίες/μεθόδους)
- Ταυτότητα (X, Y, Z)

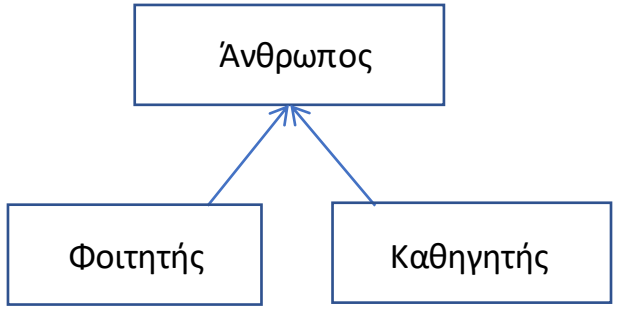
# Ενθυλάκωση (encapsulation)



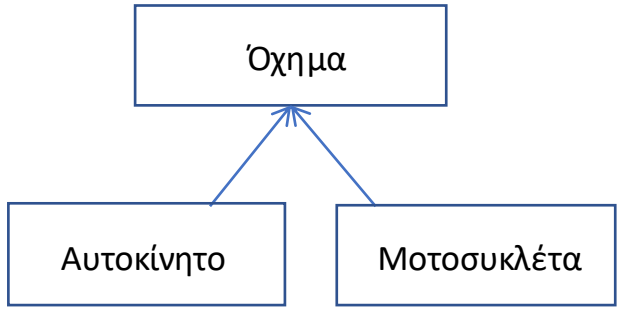

- Ένα αντικείμενο περιέχει δεδομένα και μεθόδους
- Οι λεπτομέρειες της υλοποίησης του αντικειμένου είναι κρυμμένες από το χρήστη

# Κληρονομικότητα

- Οι κλάσεις μας επιτρέπουν να ορίσουμε μια **ιεραρχία**
  - Π.χ., και ο **Φοιτητής** και ο **Καθηγητής** ανήκουν στην κλάση **Άνθρωπος**
  - Η κλάση **Αυτοκίνητο** ανήκει στην κλάση **Όχημα** η οποία περιέχει και την κλάση **Μοτοσυκλέτα**




- Οι κλάσεις πιο χαμηλά στην ιεραρχία κληρονομούν χαρακτηριστικά και συμπεριφορά από τις ανώτερες κλάσεις
  - Όλοι οι άνθρωποι έχουν **όνομα**
  - Όλα τα οχήματα έχουν μέθοδο **drive**, **stop**



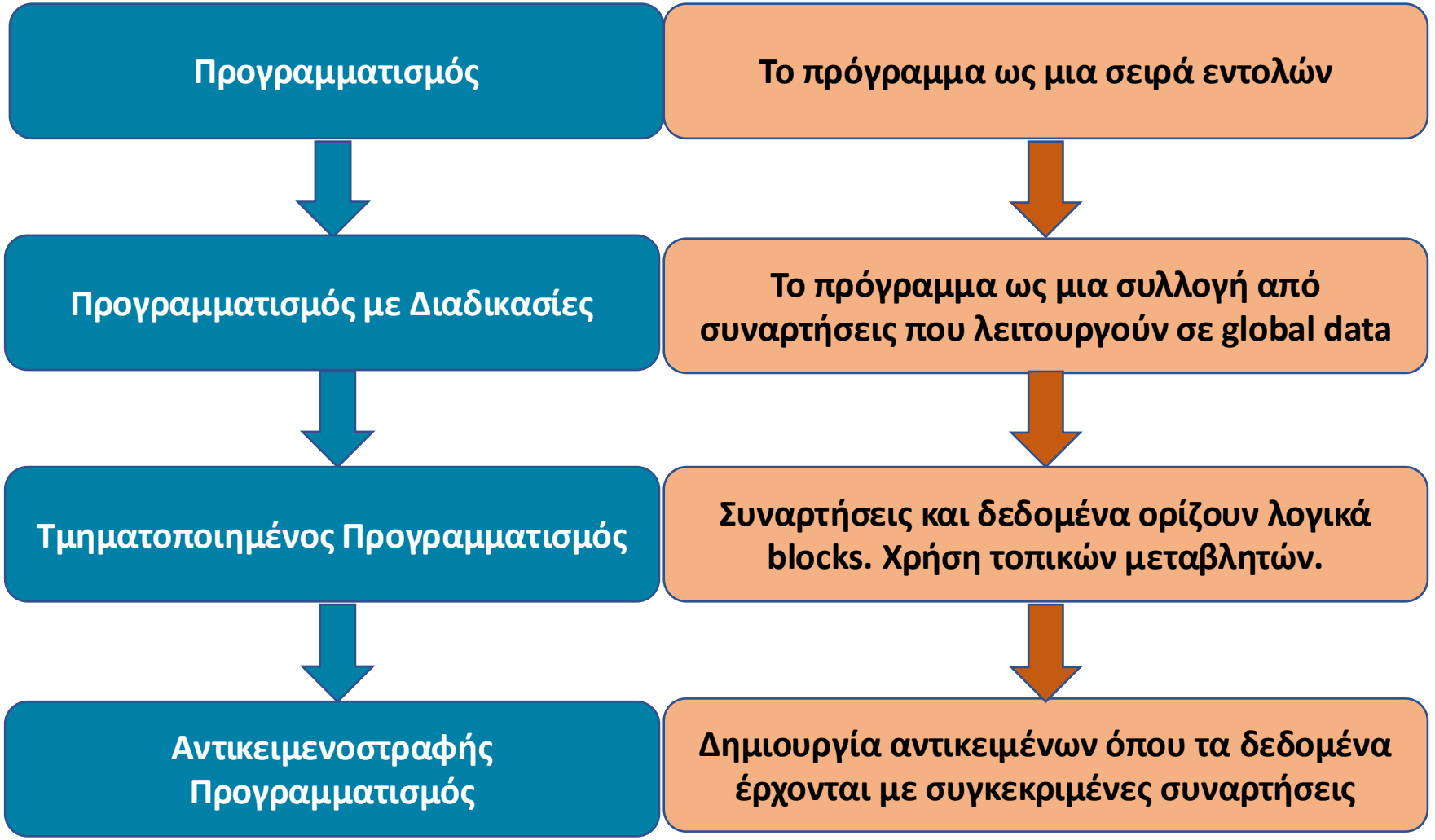
# Πολυμορφισμός

- Κλάσεις με κοινό πρόγονο έχουν κοινά χαρακτηριστικά, αλλά έχουν και διαφορές
  - Π.χ., είναι διαφορετικό το παρκάρισμα για ένα αυτοκίνητο και μια μηχανή
- Ο **πολυμορφισμός** μας επιτρέπει να δώσουμε μια **κοινή** συμπεριφορά σε κάθε κλάση (μια μέθοδο **park**), η οποία όμως **υλοποιείται διαφορετικά** για αντικείμενα διαφορετικών κλάσεων
- Μπορούμε επίσης να ορίσουμε **αφηρημένες κλάσεις**, όπου **προϋποθέτουμε** μια συμπεριφορά και αυτή πρέπει να υλοποιηθεί σε χαμηλότερες κλάσεις διαφορετικά ανάλογα με τις ανάγκες μας

# Αφηρημένοι Τύποι Δεδομένων

- Χρησιμοποιώντας τις κλάσεις μπορούμε να ορίσουμε τους δικούς μας **τύπους δεδομένων**
  - Έτσι μπορούμε να φτιάξουμε αντικείμενα με συγκεκριμένα χαρακτηριστικά και συμπεριφορά
- Χρησιμοποιώντας την κληρονομικότητα και τον πολυμορφισμό, μπορούμε να **επαναχρησιμοποιήσουμε** υπάρχοντα χαρακτηριστικά και μεθόδους

# Η εξέλιξη του προγραμματισμού



# Διαδικαστικός vs. Αντικειμενοστραφής Προγραμματισμός

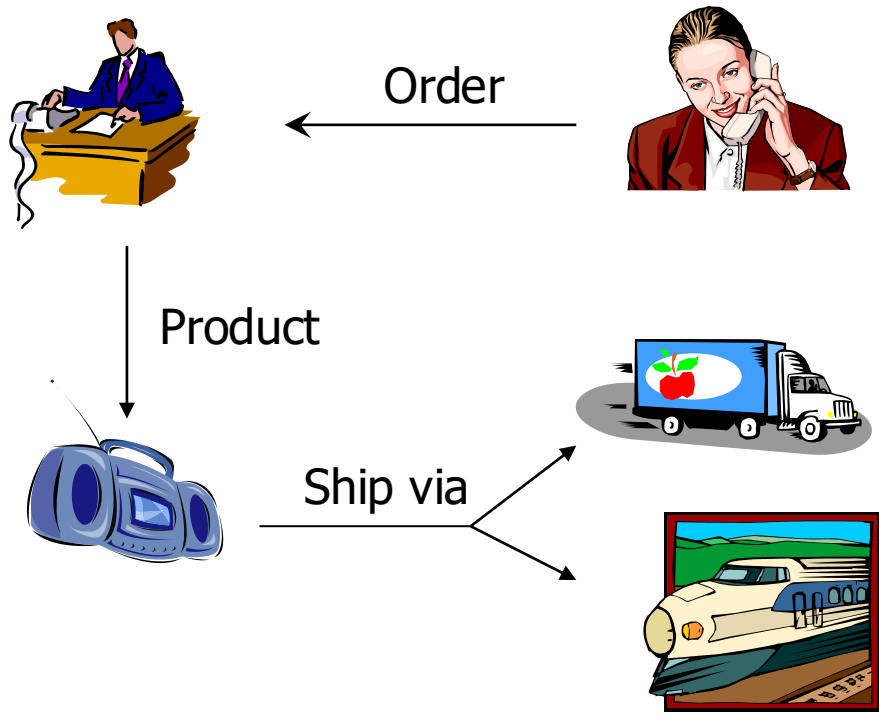
- **Διαδικαστικός**: Έμφαση στις **διαδικασίες**
  - Οι δομές που δημιουργούμε είναι για να ταιριάζουν με τις διαδικασίες
  - Οι διαδικασίες προκύπτουν από το **χώρο των λύσεων**
- **Αντικειμενοστρεφής**: Έμφαση στα **αντικείμενα**
  - Τα αντικείμενα δημιουργούνται από το **χώρο του προβλήματος**
  - Λειτουργούν ακόμη και αν αλλάξει το πρόβλημα μας

# Πλεονεκτήματα Αντικειμενοστραφούς Προγραμματισμού

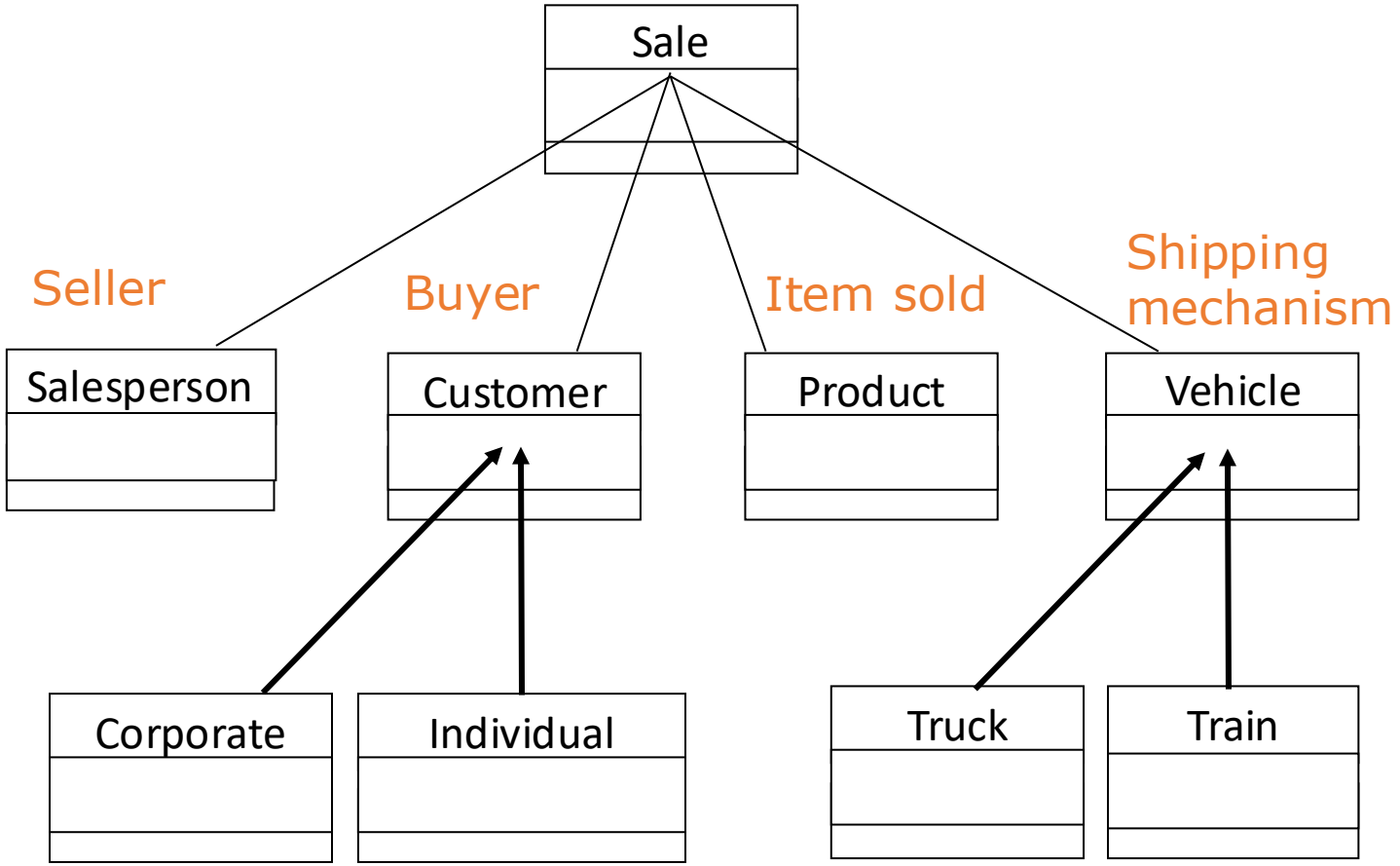
- Επειδή προσπαθεί να μοντελοποιήσει τον πραγματικό κόσμο, ο Α/Σ κώδικας είναι πιο κατανοητός
- Τα δομικά κομμάτια που δημιουργεί είναι πιο εύκολο να επαναχρησιμοποιηθούν και να συνδυαστούν
- Ο κώδικας είναι πιο εύκολο να συντηρηθεί λόγω της ενθυλάκωσης

# Παράδειγμα: Πωλήσεις

- Θέλουμε να δημιουργήσουμε λειτουργικό για ένα σύστημα το οποίο διαχειρίζεται πωλήσεις
  - **Πελάτες** κάνουν **παραγγελίες**
  - Οι **πωλητές** χειρίζονται την **παραγγελία**
  - Οι **παραγγελίες** είναι για συγκεκριμένα **προϊόντα**
  - Η **παραγγελία** αποστέλλεται με επιλεγμένο **μέσο**



# Διάγραμμα Κλάσεων



# Αλλαγή των Απαιτήσεων

